Sisteme de operare

Laborator 6

Semestrul I 2023-2024

Vlad Olaru

Laborator 6

- comunicare inter-procese (IPC)
- System V IPC
  - semafoare
  - memorie partajata

Semafoare

- primitive de sincronizare inter-procese
- folosite la protejarea sectiunilor critice
- ofera doua operatii asupra unui counter
  - *P/down/wait* – decrementeaza counter-ul
  - *V/up/signal* - incrementeaza counter-ul
  - ambele operatii sunt atomice
- accesul la o resursa partajata se obtine astfel
  - procesul incearca sa decrementeze valoarea semaforului cu 1
  - daca valoarea semaforului este > 0, accesul in sectiunea critica e permis
  - daca valoarea semaforului este 0, procesul este suspendat pana cand valoarea semaforului redevine pozitiva; in acel moment, procesul incearca din nou operatia de decrementare
- eliberarea resursei partajate
  - procesul incrementeaza valoarea semaforului cu 1
  - daca exista procese blocate in asteptarea unei valori pozitive a semaforului, acestea sunt trezite (planificate pentru rulare)

Semafoare binare vs. numaratoare

- semafoare binare
  - controleaza o singura resursa partajata
  - semaforul e initializat cu valoarea 1
- semafoare numaratoare (*counting semaphores*)
  - initializate cu o valoare pozitiva oarecare
  - valoarea initiala desemneaza nr de resurse disponibile pt. partajare

Semafoare System V

- complica paradigma generala
  - lucreaza cu set-uri de semafoare
  - crearea unui semafor (set, de fapt) e independenta de initializarea sa
    - consecinta: nu putem crea si initializa atomic un semafor
  - ca orice structura de date System V IPC continua sa existe in kernel si dupa ce nu mai exista procese care sa o utilizeze
    - daca un program termina fara sa elibereze semafoarele pe care le-a alocat, avem nevoie de o operatie de *undo*
- un semafor este reprezentat de o structura cu urmatoarele campuri
  - *semval* – valoarea semaforului, > 0
  - *sempid* – PID-ul procesului care a efectuat ultima operatie asupra semaforului
  - *semncnt* – nr proceselor care asteapta ca semval > valoarea curenta
  - *semzcnt* – nr proceselor care asteapta ca semval = 0

Crearea & intializarea semafoarelor

- crearea semafoarelor

```
int semget(key_t key, int nsems, int flag);
```

  - *key* se obtine cum am discutat anterior, cu IPC_PRIVATE sau *ftok*
  - *nsems* = nr semafoarelor din set (se foloseste valoarea 0 cand se doreste accesarea unui set de semafoare existent)
  - intoarce un *semid*

- initializarea semafoarelor

```
int semctl(int semid, int semnum, int cmd, union semun arg);
```

unde

```
union semun {
        int val;                    // pt setare valoare semafor
        struct semid_ds *buf;       // pt acces/setare structura de date asociata
                                    // semanforului
        ushort      *array;         // pt GETALL si SETALL pe setul de semafoare
}
```

Comenzi semctl

- IPC_STAT – citire structura de date semafor
- IPC_SET – setare permisiuni (UID/GID, mode)
- IPC_RMID – sterge semaforul din sistem imediat, alte procese care incearca operatii asupra semaforului primesc EIDRM
- GETVAL – intoarce valoarea *semval*
- SETVAL – seteaza valoarea *semval* la *arg.val*
- GETALL – citeste toate valorile *semval* din setul de semafoare in *arg.array*
- SETALL – seteaza toate valorile *semval* din setul se semafoare la valorile din *arg.array*
- samd

Operatii cu semafoare

- folosesc o structura de date de tipul

```
struct sembuf {
        ushort   sem_num;       // nr semafor in set: 0, 1 , …
        short    sem_op;        // operatia > 0, 0, < 0
        short    sem_flg;       // IPC_NOWAIT, SEM_UNDO
}
int semop(int semid, struct sembuf semoparray[], size_t nops);
```

- executa atomic cele *nops* operatii din *semoparray*

Semnificatia sem_op

- valoare pozitiva (eliberare resurse)
  - valoarea lui *sem_op* este adaugata la *semval*
  - pt SEM_UNDO, *sem_op* se scade din valoarea de ajustare a semaforului pt acest proces
- valoare negativa (alocare resurse)
  - *semval* >= *abs(sem_op)* => se scade *abs(sem_op)* din *semval*
    - conditia garanteaza ca semval ramane >= 0 !
    - la SEM_UNDO, *abs(sem_op)* se adauga la valoarea de ajustare a semaforului pt acest proces
  - *semval* < *abs(sem_op)* => resursele nu sunt disponibile
    - pt IPC_NOWAIT, *semop* se intoarce cu eroare setata pe EAGAIN
    - fara IPC_NOWAIT, *semncnt* se incrementeaza si procesul apelant se suspenda pana cand:

    i) *semval* >= *abs(sem_op)*, *semncnt* se decrementeaza si se scade *abs(sem_op)* din *semval;* la SEM_UNDO, *abs(sem_op)* se adauga la valoarea de ajustare a semaforului pt acest proces

    ii) semaforul este sters din sistem => *semop* intoarce ERMID

    iii) procesul primeste un semnal si handlerul de semnal se intoarce => *semncnt* se decrementeaza si *semop* se intoarce cu EINTR

Semnificatia sem_op (cont.)

- valoare nula
  - daca *semval* == 0, semop se intoarce imediat
  - daca *semval* != 0
    - pt IPC_NOWAIT, *semop* intoarce EAGAIN
    - fara IPC_NOWAIT, *semzcnt* este incrementata si procesul apelant se suspenda pana cand

    i) *semval* devine 0; *semzcnt* se decrementeaza

    ii) semaforul este sters din sistem => *semop* intoarce ERMID

    iii) procesul primeste un semnal si handlerul de semnal se intoarce => *semzcnt* se decrementeaza si *semop* se intoarce cu EINTR

Ajustarea semaforului la *exit*

- cand specificam SEM_UNDO si alocam resurse (*sem_op* < 0) kernelul memoreaza nr resurselor alocate *abs(sem_op)*
- la terminarea procesului, kernelul verifica daca exista ajustari de facut pt semafoarele procesului si le aplica
- daca setam valoarea semaforului cu *semctl* si SETVAL/SETALL valoarea de ajustare e setata pe 0 pt toate procesele

Memorie partajata

- permite mai multor procese sa partajeze o regiune de memorie
- cea mai rapida forma de IPC pentru ca nu e nevoie de copierea datelor
- dezavantaj: procesele care partajeaza regiunea de memorie trebuie sa se sincronizeze
  - altfel spus, accesele la regiunea de memorie partajata trebuie executate in sectiune critica
- se creeaza cu *shmget*

```
int shmget(key_t key, int size, int flag);
```

- *key* generat cu IPC_PRIVATE sau *ftok*
- *size* e dimensiunea regiunii de memorie partajata (*size* = 0 specifica accesul la o regiune de memorie partajata existenta)

Atasarea memoriei partajata

- un proces ataseaza o regiune de memorie partajata la spatial sau de adrese cu *shmat*

```
int shmat(int shmid, void *addr, int flag);
```

- adresa din proces la care se ataseaza regiunea de memorie depinde de valorile ultimilor doi parametri de apel

1) *addr* = 0, kernelul ataseaza regiunea de memorie la prima adresa disponibila (aceasta este metoda recomandata)
2) *addr* != 0 si SHM_RND nu apare in flag => regiunea de memorie este atasata la adresa *addr*
3) *addr* != 0 si SHM_RND setat => regiunea se ataseaza la adresa *addr* rotunjita la urmatorul multiplu al SHMLBA (*low boundary address multiple*)

- pt. SHM_RDONLY atasarea se face RO, altfel RW
- valoarea de retur a *shmat* este adresa din proces la care a fost atasata regiunea sau -1 in caz de eroare

Detasarea memoriei partajate

- detasarea regiunii de memorie de spatial de adresa al procesului se face cu *shmdt*

```
int shmdt(void *addr);
```

- Obs: detasarea nu sterge ID-ul memoriei partajate si nici structura de date aferenta care supravietuiesc pana cand se sterge zona de memorie partajata
- *addr* este valoarea returnata de un apel *shmat* anterior

Varii operatii cu memorie partajata

- executate cu *shmctl*

int shmctl(int shmid, int cmd, struct shmid_ds *buf);

Valori *cmd*: IPC_STAT – citeste in *buf* structura de date asociata in kernel regiunii de memorie

IPC_SET – seteaza permisiunile (UID/GID si *mode* in structura de data asociata in kernel regiunii de memorie)

IPC_RMID – sterge regiunea de memorie din sistem imediat, a.i orice operatie subsecventa de atasare esueaza

Obs: totusi regiunea de memorie nu dispare din kernel pana cand ultimul proces atasat la ea nu a terminat (sau se detaseaza de ea)

SHM_LOCK/SHM_UNLOCK - *pinned down memory*, memoria nu poate fi swappata pe disc (doar *root*-ul poate executa operatia de *pin down*)